

INHERITED FROM Helix Constitution

Base agent rules live in the Helix Constitution submodule at the parent project's `constitution/AGENTS.md` and the universal `constitution/Constitution.md` it references. **READ THOSE FIRST.** The base file is authoritative for any topic not covered here. Module-specific rules below extend them; they never weaken them.

Critical universal rules every CLI agent (Claude Code, Cursor, Aider, Codex, Gemini CLI) MUST honour while working in this module:

- **No bluffing.** Every PASS carries positive evidence. Constitution Â§11.4.
- **Mutation-paired gates.** Every new gate has a paired mutation proving it catches regressions. Constitution Â§1.1.
- **No guessing language** (likely, probably, maybe, seems). Constitution Â§11.4.6.
- **Credentials never tracked.** `.env` patterns git-ignored; runtime-load only. Constitution Â§11.4.10.
- **Never force-push.** Force-push requires explicit per-session authorization AND a green Â§9.1.5 post-op gate. Constitution Â§9.
- **CONTINUATION.md kept in sync** in every non-trivial commit. Constitution Â§12.10.
- **60% RAM cap.** Heavy work wrapped in bounded execution scope. Constitution Â§12.6.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

AGENTS.md â€” Project Authoritative Agent Guide

Project Agent Guidelines

Version: 3.0.0 (Updated with full architecture audit) **Date:** 2026-04-30 **Scope:** All AI agents, human contributors, and automated processes working on the project **Authority:** Derived from the parent project's AGENTS.md with project-specific enhancements

Project Overview

The project is an enterprise-grade distributed AI development platform built in Go. It enables intelligent task division, work preservation, cross-platform development workflows, and multi-provider LLM integration through a unified REST API, CLI, Terminal UI, Desktop, and Mobile client architecture.

Current Status: The `internal/` foundation is largely solid (auth, database, server, worker, task, workflow, tools, editor, notification, MCP, **verifier** are real implementations). Critical bluff and stub areas remain in select entry points and peripheral packages. All agents MUST prioritize zero-bluff implementation.

LLMsVerifier Integration Status: `internal/verifier/` package is now implemented with REST API client, two-tier cache, circuit breaker health monitor, background poller, score adapter,

and event publisher. BLUFF-002 (hardcoded CLI models) and BLUFF-004 (hardcoded external models) are FIXED. BLUFF-005 (scoring ignores verifier data) is FIXED in `ModelManager.SelectOptimalModel()`.

Key Features: - **Distributed Computing:** SSH-based worker pools with health monitoring, auto-installation, and consensus - **Multi-Provider LLM Integration:** 15+ providers (OpenAI, Anthropic, Gemini, Ollama, Azure, Bedrock, Groq, Mistral, Cohere, xAI, DeepSeek, Qwen, OpenRouter, HuggingFace, Llama.cpp) - **Development Workflows:** Automated planning, building, testing, refactoring with real shell execution - **Task Management:** Intelligent task division with priorities, dependencies, checkpointing, and Redis caching - **MCP Protocol:** Full Model Context Protocol server over WebSocket with tool dispatch - **Multi-Client Architecture:** REST API (Gin), Cobra CLI, Terminal UI (tview), Desktop (Fyne), Mobile (gomobile), WebSocket - **Memory Systems:** In-memory, filesystem, Redis, Memcached, Cognee, ChromaDB, Qdrant, Weaviate integrations - **Advanced Editor:** Multi-format code editing (diff, whole-file, search/replace, line-based) with backups - **Tools Ecosystem:** 40+ tools across filesystem, shell, web, browser, mapping, multiedit, confirmation, notebook, git - **Notifications:** Multi-channel support (Slack, Email, Telegram, Discord, Yandex Messenger, Max)

Technology Stack

Core Technologies: - **Language:** Go 1.24.0 with toolchain go1.24.9 - **Module:** `dev.helix.code` - **HTTP Framework:** Gin v1.11.0 - **Authentication:** JWT v4.5.2, bcrypt + argon2 - **Database:** PostgreSQL 15+ via pgx/v5 (optional) - **Cache:** Redis 7+ via go-redis/v9 (optional) - **Configuration:** Viper v1.21.0 - **CLI Framework:** Cobra v1.8.0 - **Testing:** Testify v1.11.1

UI Technologies: - **Desktop:** Fyne v2.7.0 - **Terminal UI:** tview v0.42.0 - **Mobile:** gomobile bindings

External Integrations: - **Browser Automation:** chromedp v0.14.2 - **Web Scraping:** goquery v1.10.3 - **Tree-sitter:** go-tree-sitter - **Identity:** Azure SDK, AWS SDK v2 - **Vector/Memory:** Cognee, ChromaDB, Qdrant, Weaviate clients - **Container Orchestration:** `digital.vasic.containers` (`vasic-digital/Containers` submodule)

Working Directory & Build System

CRITICAL: All build and test commands must be run from the `helix_code/` subdirectory, not the repository root.

```
cd <project_root>
```

Build Commands

Command	Purpose
<code>make build</code>	Build server binary to <code>bin/helixcode</code>
<code>make test</code>	Run <code>go test -v ./...</code>
<code>make test-all</code>	Run tests + coverage + benchmarks + docs
<code>make test-coverage</code>	Generate coverage report
<code>make test-benchmark</code>	Run Go benchmarks

Command

make logo-assets
make setup-deps
make fmt
make lint
make clean
make dev
make prod
make mobile
make aurora-os
make harmony-os

Purpose

Generate logo assets (required before first build)
Run go mod tidy
Run go fmt ./...
Run golangci-lint run ./...
Clean build artifacts
Start development server
Cross-platform production build
Build iOS + Android targets
Build Aurora OS target
Build Harmony OS target

Full Infrastructure Test Commands**Command**

make test-infra-up
make test-infra-down
make test-full
make test-unit-full
make test-integration-full
make test-e2e-full
make test-security-full
make test-load-full
make test-complete
make coverage-full

Purpose

Start full Docker test infrastructure
Stop full Docker test infrastructure
ALL tests with real infrastructure (zero skips)
Unit tests with real services
Integration tests with -tags=integration
E2E challenge tests via runner
Security test suite
Load tests
Sequential run of all full test types
Coverage with full infrastructure

Containerized Builds (NO Host Dependencies)**Command**

make container-builder-image
make container-build
make container-test
make container-lint
make container-shell
make container-dev-up
make container-dev-down
make container-release
./scripts/containers/build-in-container.sh

Purpose

Build the builder container image
Build application inside container
Run tests inside container
Run linter inside container
Interactive shell in builder container
Start containerized dev environment
Stop containerized dev environment
Full release build in container
Convenience wrapper script

The builder container includes: Go 1.24, gcc, postgresql-client, redis, docker-cli, golangci-lint, and all build tools. The only host requirement is Docker/Podman.

Standalone Test Scripts

Script	Purpose
<code>./run_tests.sh --unit</code>	Unit tests
<code>./run_tests.sh --integration</code>	Integration tests
<code>./run_tests.sh --e2e</code>	E2E tests
<code>./run_tests.sh --coverage</code>	Coverage analysis
<code>./run_tests.sh --security</code>	Security tests
<code>./run_all_tests.sh</code>	Orchestrates ALL suites sequentially
<code>./run_integration_tests.sh</code>	DB integration tests with Docker

Single Test Execution

```
go test -v -run TestName ./path/to/package
go test -v -tags=integration ./internal/database
cd tests/e2e/challenges && go run cmd/runner/main.go -challenge ascii-art-
```

Architecture & Code Organization

```

helix_code/
  "€"€ cmd/
    "€"€ server/main.go
    "€"€ cli/main.go
    "€"€ root.go
    "€"€ main_commands.go
    "€"€ other_commands.go
    "€"€ local-llm.go
    "€"€ local-llm-advanced.go
    "€"€ helix-config/main.go
    "€"€ security-test/main.go
    "€"€ security-fix/main.go
    "€"€ security-fix-standalone/main.go
    "€"€ performance-optimization/main.go
    "€"€ performance-optimization-standalone/main.go
    "€"€ config-test/main.go
    "€"€ internal/
      "€"€ auth/
      "€"€ llm/
      "€"€ providers/
      "€"€ compression/
      "€"€ vision/
      "€"€ provider/
      "€"€ providers/
      "€"€ worker/
      "€"€ task/
      "€"€ server/
      "€"€ database/
      "€"€ redis/
      "€"€ tools/
      # Application entry points
      # HTTP server entry point
      # Legacy flag-based CLI client
      # Cobra root command (`helix`)
      # `helix start`, `helix auto`
      # `helix server`, `helix version`
      # `helix local-llm` command tree
      # Advanced local-llm commands
      # Dedicated config management CL
      # Simulated security test runner
      # Security fix wrapper
      # Standalone security sca
      # Performance optimizer
      # Standalone p
      # Config hot-reload test utility
      # Internal packages (~40 packages)
      # JWT authentication, bcrypt/arg
      # LLM provider implementations (
      # Per-provider HTTP clients
      # Context compression
      # Vision/multimodal support
      # Provider abstractions
      # Provider management
      # SSH-based worker pool, health
      # Task queues, dependencies, che
      # Gin HTTP server, routes, middl
      # PostgreSQL pgx pool, schema in
      # go-redis wrapper with graceful
      # 40+ tool ecosystem registry

```

â",	â",	â"€â"€â"€ filesystem/	# fs_read, fs_write, fs_edit,
â",	â",	â"€â"€â"€ shell/	# shell, shell_background with
â",	â",	â"€â"€â"€ web/	# web_fetch, web_search
â",	â",	â"€â"€â"€ browser/	# browser_launch, browser_navi
â",	â",	â"€â"€â"€ multiedit/	# Transactional multi-file edi
â",	â",	â""â"€â"€ git/	# Git automation
â",	â"€â"€â"€	editor/	# Multi-format code editing with
â",	â"€â"€â"€	memory/	# Memory providers (in-mem, file
â",	â"€â"€â"€	cognee/	# Cognee.ai memory integration
â",	â"€â"€â"€	context/	# Hierarchical context managemen
â",	â"€â"€â"€	notification/	# Multi-channel notification eng
â",	â"€â"€â"€	mcp/	# Model Context Protocol WebSock
â",	â"€â"€â"€	workflow/	# Development workflow execution
â",	â"€â"€â"€	config/	# Viper-based configuration mana
â",	â"€â"€â"€	event/	# Pub/sub event bus
â",	â"€â"€â"€	logging/	# Structured logging wrapper
â",	â"€â"€â"€	monitoring/	# Metric collection framework
â",	â"€â"€â"€	security/	# Security scanning (stubbed)
â",	â"€â"€â"€	session/	# Development session management
â",	â"€â"€â"€	agent/	# Agent orchestration
â",	â"€â"€â"€	project/	# Project management
â",	â"€â"€â"€	rules/	# Rules engine
â",	â"€â"€â"€	hooks/	# Hook system
â",	â"€â"€â"€	focus/	# Focus chain management
â",	â"€â"€â"€	template/	# Template system
â",	â"€â"€â"€	persistence/	# State persistence
â",	â"€â"€â"€	deployment/	# Deployment management
â",	â"€â"€â"€	discovery/	# Service/model discovery
â",	â"€â"€â"€	hardware/	# Hardware abstraction
â",	â"€â"€â"€	repomap/	# Repository mapping
â",	â"€â"€â"€	version/	# Version management
â",	â"€â"€â"€	fix/	# Security fix engine
â",	â"€â"€â"€	performance/	# Performance optimization
â",	â"€â"€â"€	testutil/	# Test utilities
â",	â""â"€â"€	mocks/	# Shared mocks
â",	â"€â"€â"€	applications/	# Platform-specific applications
â",	â"€â"€â"€	desktop/	# Fyne desktop app
â",	â"€â"€â"€	terminal-ui/	# tview terminal UI
â",	â"€â"€â"€	android/	# Android app
â",	â"€â"€â"€	ios/	# iOS app
â",	â"€â"€â"€	aurora-os/	# Aurora OS client
â",	â""â"€â"€	harmony-os/	# Harmony OS client
â",	â"€â"€â"€	api/	# OpenAPI specification
â",	â""â"€â"€	openapi.yaml	# Full REST API spec (OpenAPI 3.
â",	â"€â"€â"€	config/	# Configuration files
â",	â"€â"€â"€	config.yaml	# Primary application config
â",	â"€â"€â"€	production-config.yaml	# Enterprise production config
â",	â"€â"€â"€	minimal-config.yaml	# Minimal test config (DB/Redis
â",	â"€â"€â"€	test-config.yaml	# Test-specific config
â",	â"€â"€â"€	working-config.yaml	# Working variant
â",	â"€â"€â"€	azure_example.yaml	# Azure-specific example

```

â",    â""â"€â"€ model-aliases.example.yaml# Model alias examples
â",
â"œâ"€â"€ tests/                                # New test framework
â",    â"œâ"€â"€ e2e/challenges/                # Challenge-based E2E tests
â",    â""â"€â"€ automation/                    # Hardware automation tests
â",
â"œâ"€â"€ test/                                # Legacy/parallel test suites
â",    â"œâ"€â"€ integration/                  # Integration tests
â",    â"œâ"€â"€ e2e/                          # Legacy E2E tests
â",    â"œâ"€â"€ automation/                  # Provider automation tests
â",    â""â"€â"€ load/                        # Load tests
â",
â"œâ"€â"€ benchmarks/                        # Performance benchmarks
â"œâ"€â"€ security/                          # Security tests
â"œâ"€â"€ standalone_tests/                  # Standalone CLI tests
â"œâ"€â"€ docker/                            # Docker assets and extended compo
â"œâ"€â"€ scripts/                          # Build and deployment scripts
â""â"€â"€ assets/                          # Logo and image assets

```

Verified Real Implementations

AUTH-001: Authentication System (VERIFIED REAL)

File: `internal/auth/auth.go` (~470 lines) **Assessment:** Production-ready - User registration with validation - Password hashing with bcrypt + argon2 fallback - JWT token generation and verification (JWT v4) - Session management with crypto-random tokens - Constant-time comparison for timing attack prevention - Full test coverage in `internal/auth/auth_test.go` (~777 lines)

DB-001: Database Layer (VERIFIED REAL)

File: `internal/database/database.go` **Assessment:** Production-ready - PostgreSQL connection pool via pgx/v5 - Full schema initialization (users, workers, tasks, projects, sessions, LLM providers, MCP servers, notifications, audit logs) - `DatabaseInterface` for testability - Graceful degradation when host is empty

SRV-001: HTTP Server (VERIFIED REAL)

File: `internal/server/server.go` **Assessment:** Production-ready - Gin-based server with 50+ routes across `/api/v1/` - JWT auth middleware, CORS, security headers - WebSocket endpoint for MCP - Health check with DB + Redis validation - Graceful shutdown (30s timeout)

LLM-001: LLM Providers (VERIFIED REAL)

File: `internal/llm/` (~5000+ lines across providers) **Assessment:** Real HTTP clients - `AnthropicProvider` (~752 lines): Full SSE streaming, prompt caching, extended thinking, tool calls - `OpenAIProvider` (~431+ lines): Full HTTP API client - `ModelManager`: Multi-provider orchestration, selection strategy, fallback chain - 16 provider subdirectories with real HTTP implementations - **Note:** The `internal/llm/` package is genuine. Bluff areas are at `cmd/cli/main.go` only.

WRK-001: Worker Pool (VERIFIED REAL)

File: `internal/worker/` (~800+ lines) **Assessment:** Real distributed worker management - WorkerManager: Register, heartbeat, assign tasks, complete tasks - SSH config parsing, capability matching, resource tracking - Health checks with TTL

TSK-001: Task Management (VERIFIED REAL)

File: `internal/task/` (~1000+ lines) **Assessment:** Real task lifecycle - Priority queues, dependency validation, checkpointing - Redis caching with graceful degradation - Retry logic and cleanup

WFL-001: Workflow Engine (VERIFIED REAL)

File: `internal/workflow/` (~1100+ lines) **Assessment:** Real shell execution - Executor dispatches to real `exec.CommandContext()` calls - Security filtering via `isDangerousCommand()` (rm, dd, mkfs, fork bombs, etc.) - LLM integration with real `LLMRequest` - Supports Go, Node, Python, Rust project types

TOO-001: Tools Ecosystem (VERIFIED REAL)

File: `internal/tools/` (~2000+ lines) **Assessment:** Real tool registry - 8 categories: filesystem, shell, web, browser, mapping, multiedit, confirmation, notebook - Real `chromedp` browser automation - Transactional multi-file editing

EDT-001: Code Editor (VERIFIED REAL)

File: `internal/editor/` (~600+ lines) **Assessment:** Real file I/O - Diff, whole-file, search/replace, line-based editors - Automatic file backup with `io.Copy` - `EditApplier` / `EditValidator` interfaces

NOT-001: Notification Engine (VERIFIED REAL)

File: `internal/notification/` (~800+ lines) **Assessment:** Real HTTP/SMTP calls - Slack (webhook HTTP POST), Email (SMTP via `net/smtp`), Telegram (Bot API), Discord (webhook) - Yandex Messenger (OAuth API), Max (enterprise API) - Rate limiting, retry, queue, metrics

MCP-001: MCP Protocol Server (VERIFIED REAL)

File: `internal/mcp/` (~400+ lines) **Assessment:** Real WebSocket server - gorilla/websocket concurrent session handling - JSON-RPC-like message format - Tool execution dispatch

CFG-001: Configuration Management (VERIFIED REAL)

File: `internal/config/` (~1700+ lines) **Assessment:** Full Viper integration - Environment variable binding (`HELIX_*`) - Config file search (`.`, `$HOME/.helixcode`, `/etc/helixcode`) - Validation rules, default config creation - `ConfigManager` for load/save/merge

QA-001: HelixQA Integration (VERIFIED REAL)

Files: internal/helixqa/, internal/server/qa_handlers.go, applications/terminal-ui/main.go **Assessment:** Full embedded QA engine with real session lifecycle - Engine struct manages QA sessions with map + sync.RWMutex - StartSession(), CancelSession(), GetSession(), ListSessions() with real state tracking - REST API: POST /api/v1/qa/session, GET /api/v1/qa/session/:id/status, GET /api/v1/qa/session/:id/report, GET /api/v1/qa/session/:id/screenshot/:name, DELETE /api/v1/qa/session/:id - CLI flags: --qa-run, --qa-list, --qa-report, --qa-screenshot, --qa-cancel - TUI dashboard with session table, stats panel, refresh/cancel actions - Screenshot pipeline: 8 platform engines (Linux, Web, iOS, Android, CLI, TUI, macOS, Windows) - Tests: internal/helixqa/wrapper_test.go, internal/server/qa_handlers_test.go, pkg/screenshot/*_test.go

Verified Bluff & Stub Areas (MUST FIX)

BLUFF-001: LLM Generation is Simulated in Legacy CLI (CRITICAL) â€” FIXED

File: cmd/cli/main.go lines ~236-284 **Evidence:** Previously returned `fmt.Sprintf("Generated response for: %s...", prompt)` without calling any provider. **Fix:** `handleGenerate()` now constructs a real `llm.LLMRequest` with user messages and calls `provider.Generate() / provider.GenerateStream()`. Errors are propagated to the user if the provider is unavailable. **Verification:** `go build -tags nogui ./cmd/cli/` compiles; provider call is real (returns error if Ollama/etc. is not running). **Fix Priority:** P0 â€” RESOLVED

BLUFF-002: Model Listing is Hardcoded in Legacy CLI (CRITICAL) â€” FIXED

File: cmd/cli/main.go lines ~101-128 **Evidence:** Previously only 3 hardcoded models. No dynamic discovery. **Fix:** Replaced with verifier-aware `handleListModel()` that queries `LLMsVerifier` adapter first, falls back to provider discovery, then to constitutional `FallbackModels` (7 models with scores and verification status). **Verification:** `go test -v ./internal/verifier/...` passes; `go build ./cmd/cli/...` compiles. **Fix Priority:** P0 â€” RESOLVED

BLUFF-003: Command Execution is Simulated in Legacy CLI (HIGH) â€” FIXED

File: cmd/cli/main.go lines ~310-324 **Evidence:** Previously printed the command and slept for 1 second without executing anything. **Fix:** `handleCommand()` uses `exec.CommandContext(ctx, "sh", "-c", command)` with real `os.Stdout / os.Stderr` redirection. Exit codes are reported. **Verification:** `go build -tags nogui ./cmd/cli/` compiles. **Fix Priority:** P0 â€” RESOLVED

STUB-001: Security Scanning is Simulated

File: `internal/security/security.go` (~132 lines) **Evidence:** `ScanFeature()` contains explicit `“Simulate security scanning logic”` comment. Always returns `Success=true, Score=95` with empty issues. **Fix Priority:** P1

STUB-002: Memory Redis/Memcached Providers Store Locally

File: `internal/memory/` (~1800+ lines) **Evidence:** `RedisMemoryProvider` and `MemcachedMemoryProvider` store data in local maps with comments like `“Redis client would be used in production.”` Connection config is parsed but not used. **Fix Priority:** P2

STUB-003: Security-Test Entry Point is Entirely Simulated

File: `cmd/security-test/main.go` **Evidence:** Hardcoded list of 12 simulated security tests. `simulateSecurityScan()` returns pre-canned issue lists per category. **Fix Priority:** P2

STUB-004: Several helix Subcommands are Print-Only

File: `cmd/other_commands.go` **Evidence:** `server`, `generate`, `test`, `worker`, `notify` commands are stubbed (print placeholder messages). **Fix Priority:** P2

STUB-005: Several helix-config Subcommands are Placeholders

File: `cmd/helix-config/main.go` **Evidence:** Many `template/history/schema` subcommands print placeholder messages. **Fix Priority:** P3

BLUFF-004: LLMsVerifier Integration is Stubbed or Bypassed (CRITICAL)

File Pattern: `internal/verifier/*.go` containing empty structs, `// TODO`, or methods that return hardcoded data instead of calling the verifier. **Evidence:** - `VerificationService` methods return hardcoded `VerificationResult{OverallScore: 8.5}` instead of querying the verifier database - `ModelDiscoveryService` returns an empty slice instead of calling provider APIs - The verifier client returns fallback models without attempting a real HTTP call **Fix Priority:** P0 - Immediate **Verification Command:**

```
make test-verifier-integration
# This MUST pass with real verifier data, not mocked scores
```

BLUFF-005: Provider Discovery Uses Hardcoded Env Var Names (HIGH)

File Pattern: `internal/verifier/startup.go` or provider adapter files containing hardcoded strings like `"OPENAI_API_KEY"` without checking `SupportedProviders[provider].EnvVars`. **Fix Priority:** P1 - High

BLUFF-006: Model Capabilities Are Hardcoded (HIGH)

File Pattern: `internal/llm/*.go` containing `SupportsToolUse: true` as a struct literal for specific models, or `Provider.GetCapabilities()` returning a static slice. **Fix Priority:** P1 - High **Constitutional Impact:** Violates CONST-041 (MCP/LSP/ACP/Embedding/RAG/Skills/Plugins Integration Mandate).

BLUFF-007: Test Claims Integration But Uses Mocked Verifier (CRITICAL)

File Pattern: `*_test.go` files with `testify/mock` or `testMode: true` in non-unit test files. **Fix Priority:** P0 - Immediate **Constitutional Impact:** Violates CONST-038 (Model Provider Anti-Bluff Guarantee) and CONST-035 (Zero-Bluff Testing).

BLUFF-008: Scoring Weights Do Not Sum to 1.0 (MEDIUM)

File Pattern: `configs/verifier.yaml` or `internal/verifier/config.go` where scoring weights are misconfigured. **Fix Priority:** P2 - Medium

BLUFF-009: /metrics Endpoint Returns Hardcoded Zeros (CRITICAL) â€” FIXED

File: `internal/server/handlers.go` lines ~834-855 **Evidence:** All dynamic metrics (goroutines, memory, database connections) were hardcoded to 0. **Fix:** `getMetrics()` now calls `runtime.ReadMemStats()`, `runtime.NumGoroutine()`, and `s.db.Pool.Stat()` to return real values. **Fix Priority:** P0 â€” RESOLVED

BLUFF-010: Multi-Edit Conflict Detection is a No-Op (HIGH) â€” FIXED

File: `internal/tools/multiedit/transaction.go` lines ~352-369 **Evidence:** `detectFileConflict()` always returned `nil`, `nil` with comment â€œFor now, weâ€™ll assume no conflicts.â€” **Fix:** Implemented real conflict detection â€” reads the file from disk, computes SHA-256, and compares against the `Checksum` field. Returns `ConflictModified` or `ConflictDeleted` when appropriate. **Fix Priority:** P1 â€” RESOLVED

Configuration Management

Primary Configuration

Main config at `config/config.yaml`:

```
server:
  address: "0.0.0.0"
  port: 8080
  read_timeout: 30
  write_timeout: 30
  idle_timeout: 300
  shutdown_timeout: 30

database:
  host: "" # Empty string disables PostgreSQL
  port: 5432
  user: "helix"
  password: "${HELIX_DATABASE_PASSWORD}"
  dbname: "helixcode_prod"
  sslmode: "disable"

redis:
  host: "redis"
```

```
port: 6379
password: "${HELIX_REDIS_PASSWORD}"
db: 0
enabled: true
```

```
auth:
  jwt_secret: "${HELIX_AUTH_JWT_SECRET}"
  token_expiry: 86400
  session_expiry: 604800
  bcrypt_cost: 12
```

```
workers:
  health_check_interval: 30
  health_ttl: 120
  max_concurrent_tasks: 10
```

```
tasks:
  max_retries: 3
  checkpoint_interval: 300
  cleanup_interval: 3600
```

```
llm:
  default_provider: "local"
  max_tokens: 4096
  temperature: 0.7
  timeout: 30
  max_retries: 3
  providers:
    <name>:
      type: <provider-type>
      endpoint: <url>
      enabled: true
      parameters:
        timeout: 30.0
        max_retries: 3
        streaming_support: true
        api_key: ""
  selection:
    strategy: "performance"
    fallback_enabled: true
    health_check_interval: 30
```

```
logging:
  level: "info"
  format: "text"
  output: "stdout"
```

```
notifications:
  enabled: true
  rules:
    - name: "...".
      condition: "type==error"
      channels: ["slack", "email"]
      priority: urgent
```

```
    enabled: true
channels:
  slack: { enabled, webhook_url, channel, username, timeout }
  telegram: { enabled, bot_token, chat_id, timeout }
  email: { enabled, smtp: { server, port, username, password, tls }, rec
  discord: { enabled, webhook_url, timeout }
```

Environment Variables

Required for Production: - HELIX_DATABASE_PASSWORD - HELIX_AUTH_JWT_SECRET
- HELIX_REDIS_PASSWORD

LLM Provider Keys (as needed): - OPENAI_API_KEY, ANTHROPIC_API_KEY,
GEMINI_API_KEY, XAI_API_KEY, DEEPSEEK_API_KEY, GROQ_API_KEY,
MISTRAL_API_KEY, COHERE_API_KEY, AZURE_OPENAI_API_KEY,
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY

Notification Integrations: - HELIX_SLACK_WEBHOOK_URL -
HELIX_TELEGRAM_BOT_TOKEN, HELIX_TELEGRAM_CHAT_ID -
HELIX_EMAIL_SMTP_SERVER, HELIX_EMAIL_USERNAME, HELIX_EMAIL_PASSWORD
- HELIX_DISCORD_WEBHOOK_URL

Testing Strategy

Test Categories

1. **Unit tests:** Mocks allowed, *_test.go, -short flag
2. **Contract tests:** Real API schemas, no mocks
3. **Component tests:** Real subsystems wired together
4. **Integration tests:** Full app with real dependencies (-tags=integration)
5. **E2E challenges:** Complete user workflows against real LLM APIs
6. **Security tests:** OWASP compliance
7. **Performance tests:** Benchmarks
8. **Automation tests:** Provider/hardware automation (-tags=automation)
9. **Load tests:** Stress testing

Anti-Bluff Testing Rules

- Unit tests: Mocks OK
- **ALL other tests: Real infrastructure ONLY**
- Every PASS guarantees **Quality + Completion + Usability**
- Challenges fail on simulated/stubbed behavior
- No bare t.Skip() without SKIP-OK: #<ticket> marker

Docker Test Infrastructure

- docker-compose.test.yml: PostgreSQL 16, Redis 7, Memcached, Cognee, ChromaDB, Qdrant, Ollama, Prometheus, Grafana
- docker-compose.full-test.yml: Complete stack with mock-LLM server, Selenium, ChromeDP, SSH server + 3 workers, Cognee, Weaviate, mock-Slack, multicast router

Challenge Framework (tests/e2e/challenges/)

The most rigorous test system validates the project by having it **generate real projects** and testing them: - **Challenge Definitions:** JSON specs (ASCII art generator, CLI task manager, JSON validator, notes API, tic-tac-toe TUI, URL shortener) - **Execution Flow:** Load spec → Call real LLM API → Parse generated code → Compile → Test → Runtime validation - **Validation Layers:** Directory structure, code quality, compilation, testing, functionality, runtime validation with diverse data - **Test Matrix:** Supports CLI, TUI, REST, WebSocket interfaces across 15+ providers and worker pool distributions

Test Scripts Summary

```
# Basic
cd <project_root> && make test

# Full infrastructure (recommended for validation)
make test-infra-up
make test-complete
make test-infra-down

# Individual categories
make test-unit-full
make test-integration-full
make test-e2e-full
make test-security-full
make test-load-full

# Legacy scripts
./run_tests.sh --all
./run_all_tests.sh
./run_integration_tests.sh
```

Docker Deployment

Production (docker-compose.yml)

Services: helixcode-server (8080, 2222), postgres:15, redis:7, nginx (80, 443), prometheus (9090), grafana (3000)

Quick Start

```
cd <project_root>
cp .env.example .env
# Edit .env with secure passwords
docker compose up -d
docker compose ps
curl http://localhost/health
```

Other Compose Files

File	Purpose
<code>docker-compose-simple.yml</code>	Minimal dev (postgres + redis only)
<code>docker-compose.test.yml</code>	Integration/E2E testing stack
<code>docker-compose.full-test.yml</code>	Zero-skip full test infrastructure
<code>docker-compose.aurora-os.yml</code>	Security-focused Aurora OS platform
<code>docker-compose.harmony-os.yml</code>	Distributed Harmony OS platform
<code>docker-compose.specialized-platforms.yml</code>	Combined Aurora + Harmony
<code>docker/docker-compose.yml</code>	Extended full-stack with Milvus, Elasticsearch, MLflow, Jaeger, Jupyter, Portainer

Deployment Patterns

- Healthchecks on every service
 - Docker profiles: `monitoring`, `distributed`, `with-redis`, `production`, `dev`, `server`
 - Isolated bridge networks per deployment
 - Named persistent volumes for all stateful services
 - `.env` file for secrets
-

Code Style & Development Conventions

Go Conventions

- Standard Go formatting: `go fmt ./...`
- Linting: `golangci-lint run ./...` (timeout 10m in CI)
- Vet: `go vet ./...`
- Table-driven tests with `t.Run()` subtests
- Build tags for integration/automation tests: `//go:build integration`

Project Conventions

- **Always work from `helix_code/ subdirectory`**
- **Generate logo assets before first build:** `make logo-assets`
- **Database/Redis optional:** Disable by setting `database.host: ""`
- **Environment variables override config file**
- Use `internal/` for all core packages; no `pkg/` directory in active use
- Error handling: explicit, no silent failures
- Concurrent access: use `sync.RWMutex` or channel patterns

API Conventions

- REST API documented in `api/openapi.yml` (OpenAPI 3.0.3)

- Base path: `/api/v1`
 - Authentication: Bearer JWT via `Authorization` header
 - Health endpoint: GET `/health` (no auth required)
-

Security Considerations

Verified Security Features

- Password hashing: bcrypt (cost 12) with argon2 fallback
- JWT with constant-time comparison
- CORS middleware, security headers (X-Frame-Options, CSP, HSTS)
- Rate limiting support in production config
- Session timeout, concurrent session limits, IP binding options
- Workflow `isDangerousCommand()` filter blocks `rm`, `dd`, `mkfs`, fork bombs, etc.
- Input validation in auth and server packages

Security Testing

- `security/security_test.go`: OWASP Top 10, SAST, DAST, credential scanning, TLS enforcement, input validation (path traversal, XSS, SQL injection, command injection, SSRF)
- File permission checks (0600 for configs)

Known Security Stubs

- `internal/security/security.go`: Simulated scanning (always returns clean)
- `cmd/security-test/main.go`: Entirely simulated security tests

Production Hardening

- Use `HELIX_AUTH_JWT_SECRET` with high entropy
 - Enable PostgreSQL SSL in production
 - Enable Redis authentication
 - Configure CORS `allowed_origins` explicitly
 - Enable audit logging
 - Set `bcrypt_cost: 14` in production
-

Free AI Providers

- **XAI (Grok)**: `grok-3-fast-beta`, `grok-3-mini-fast-beta`
 - **OpenRouter**: Free models from various providers
 - **GitHub Copilot**: `gpt-4o`, `claude-3.5-sonnet` (with subscription)
 - **Qwen**: 2,000 requests/day free tier
-

Common Issues

1. **Build fails**: Run `make logo-assets` then `make build`
2. **Database errors**: Check `HELIX_DATABASE_PASSWORD`

3. **Worker SSH failures:** Verify SSH key authentication
 4. **LLM timeouts:** Check provider status and config
 5. **Redis connection failures:** Check `HELIX_REDIS_PASSWORD` and `redis.enabled`
 6. **Test skips:** Ensure `SKIP-OK: #<ticket>` marker is present for any intentional skips
-

Resources & References

- **Constitution:** `CONSTITUTION.md`
 - **CLAUDE.md:** `CLAUDE.md`
 - **Gap Analysis:** `HELIXCODE_GAP_ANALYSIS.md`
 - **Zero-Bluff Plan:** `HELIXCODE_ZERO_BLUFF_PLAN.md`
 - **Testing Strategy:** `ANTI_BLUFF_TESTING_STRATEGY.md`
 - **OpenAPI Spec:** `helix_code/api/openapi.yaml`
 - **Docker Guide:** `helix_code/DOCKER_DEPLOYMENT.md`
-